

SPECTRA CHAT

主な使用技術/プラグイン

フロントエンド



バックエンド



その他プラグイン/開発支援ツール



環境構築

- docker 環境を準備

[docker 公式スタートガイドページ](#)

- node 環境を準備

- node のバージョン管理は [volta](#) がおすすめ。指定バージョンの node 環境、パッケージマネージャーを用意してください

```
node v24.12.0
```

```
pnpm v10.22.0
```

- ビルドする際、ファイルの大文字小文字の区別をしっかりと管理しないと表示されない場合があるので注意してください

- 初期データ取得

こちらから取得したものを02_src\SPECTRA_CHAT\migrationに配置

- env ファイル追加

- 02_src\SPECTRA_CHAT\backend 直下に.envファイルを追加。このとき、.env ファイルの形式は、env.txtを参考にすること。

```
//.envの例
GEMINI_API_KEY=XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

- アプリケーションを立ち上げる方法

- 02_src\SPECTRA_CHAT に移動

```
docker compose up --build
```

```
python pipeline.py
```

- 02_src\SPECTRA_CHAT\frontend に移動

```
//package.jsonの中身を適用
pnpm install
```

```
//開発サーバーを立ち上げ
pnpm run dev
```

```
//ローカルネットワーク経由で開発サーバーを確認できます。フロント側の`.env`ファイルを`localhost`から一時的に変更してください。
pnpm run dev --host
```

<http://localhost:3000/>をブラウザで開く

ディレクトリ構成

```
.
├── 01_docs //主に資料をここに置く
├── 02_src  //本番プロジェクトをここに置く
│   └── SPECTRA_CHAT //アプリケーションディレクトリ
```

```

├── frontend
│   ├── src
│   │   ├── presentation
│   │   │   ├── atoms //最小単位のUIコンポーネント htmlのタグと対応 どのページでも使
│   │   │   │   います
│   │   │   ├── molecules //atomsを組み合わせたもの どのページでも使い回す
│   │   │   ├── organisms // ページ単位で管理するUIコンポーネント
│   │   │   │   ├── homeSite // 【ページ名】Site
│   │   │   │   └── __tests__ //テストコード格納フォルダ use【機能名】
│   │   ├── App.test.ts
│   │   ├── core //usecase アプリケーションロジック use【機能名】App.ts
│   │   └── ui //ロジック(機能)と接続するUI 【ページ名】SiteMain.vue、
│   │       【機能名】App.vue
│   └── templates //ロジック接続後のUI 基本的にページを表示するだけ 【ページ
│       名】
│   ├── domain
│   │   ├── interfaces // インターフェイスをまとめて保存
│   │   ├── types //型をまとめて保存
│   │   └── params //固定値をまとめて保存
│   ├── infrastructure
│   │   └── stores //状態管理ストア
│   ├── App.vue // UIのエントリーポイント AtomicDesignのPagesにあたる
│   ├── main.ts //ロジックのエントリーポイント
│   └── style.css //CSSのエントリーポイント
│   ├── public //画像、アイコン、素材を置く
│   └── package.json //フロントエンドで使用しているライブラリの管理を行う
├── backend
│   ├── main.py //エントリーポイント
│   ├── controller //httpのエンドポイント兼MCP
│   ├── infrastructure //外部ライブラリを使用したロジックを隔離
│   └── domain //抽象基底クラスを保管
├── migration //DBに初期投入するデータを保管
├── pipeline.py //DBに初期投入するスクリプト
├── compose.yaml //アプリケーション全体のエントリーポイント
└── 03_prototype //プロトタイププロジェクトをここに置く

```

アプリケーション全体のアーキテクチャ

- モノリシックなソフトウェア構成
- Docker コンテナごとに分けて実装
- 要素間は port 番号を指定して通信する
- backend は、<http://localhost:4000/docs>にてアクセスを swagger で検証できる

```

flowchart TD
    compose[エントリーポイント  
compose.yaml]
    frontend[frontendコンテナ Vue  
port3000]
    backend[backendコンテナ FastAPI  
port4000]
    postgis[postGISコンテナ (DB) PostGIS]

```

```
port5432]
pipeline[pipeline.py
postGISにmigrationフォルダ内データを一括投入するスクリプト]

compose --> frontend
compose --> backend
compose --> postgis
pipeline --> postgis
```

フロントエンドのアーキテクチャ

- DI(依存性の注入)によって、依存性逆転の原則を保つ
- usecase（純粋な内部ロジック）と infrastructure（外部ロジック）がインターフェースを介してやり取りを行うことで、内部ロジックをクリーンに保つことができる。
- コメントは TSDOC 方式を推奨
- テストコードには vitest を用いる
- インデントには prettier を、ビルドチェックに eslint を用いる

flowchart TD

```
%% --- 層の定義 ---
subgraph UI層["UI（プレゼンテーション層）"]
    UI["コンポーネント / ページ"]
end

subgraph Usecase層["Usecase（アプリケーション層）"]
    Usecase["ビジネスロジック / ユースケース"]
end

subgraph Domain層["Domain（ドメイン層 / interface定義）"]
    Domain["IMapInstance / IDialogStateStore など"]
end

subgraph Infrastructure層["Infrastructure（インフラ層）"]
    Infrastructure["地図ライブラリ / HTTP通信 / 状態管理（Pinia）など"]
end

%% --- 関係性の定義 ---
UI -->|引数にインターフェースをいれて、DIを行う| Usecase
Usecase -->|インターフェースに依存| Domain
Infrastructure -->|インターフェースを実装| Domain
UI ---|ユースケースの結果を描画| Usecase
```

AtomicDesign による UI コンポーネントマネジメント

- コンポーネントを階層的に管理することで、UI の拡張性を高める
- コンポーネントは storybook で管理する

flowchart TD

```

%% --- Atomic Design レイヤー構造 ---
subgraph AtomicDesign["Atomic Design によるUIコンポーネント構造"]

    A[Atom
    (UI部品の最小単位
    例: Button, Input, Icon) ]
    B[Molecule
    (Atomの組み合わせ
    例: Form, Card, ModalHeader) ]
    C[Organism
    (Atoms/Moleculesを配置し
    ロジックを持つ単位
    ページ相当の構成要素) ]
    D[Template
    (状況に応じてOrganismを切り替える
    例: デバイス別・テーマ別表示) ]
    E[Page
    実際に表示される画面 (ルーティングに対応し
    特定のTemplateを表示する) ]
end

%% --- 関係性 ---
A --> B
B --> C
C --> D
D --> E

%% --- ロジック・ユースケース層との関係 ---
subgraph Logic["ロジック層 / クリーンアーキテクチャとの接続"]
    Usecase[UseCase
    (ビジネスロジック / アプリケーション層) ]
    Domain[Domain
    (インターフェース / エンティティ定義) ]
    Infra[Infrastructure
    (HTTP / Map / DB / Store) ]
end

%% --- 依存関係の流れ ---
C -->|UIイベントで呼び出し| Usecase
Usecase -->|インターフェースに依存| Domain
Infra -->|インターフェースを実装| Domain
C ---|UI更新| Usecase

```

バックエンドのアーキテクチャ

- 基本的にはフロントエンドと同じ。
- ruff コマンドを使ってインデントを修正できる

```
uv run ruff format
```

postgres に入るコマンド

```
docker exec -it postgis_container psql -U docker -d postgres
```

テーブル名一覧取得

```
\dt
```

flowchart TD

%% --- 層の定義 ---

ENTRY["エントリーポイント main.py"]

subgraph Usecase層["Controller（アプリケーション層）"]
Usecase["リクエストコントローラー"]
end

subgraph Domain層["Domain（ドメイン層 / 抽象基底クラス）"]
Domain["共通する抽象基底クラスを継承"]
end

subgraph Infrastructure層["Infrastructure（インフラ層）"]
Infrastructure["外部ロジック用リポジトリ"]
end

%% --- 関係性の定義 ---

ENTRY -->|呼び出し| Usecase

Usecase -->|呼び出し| Infrastructure

Usecase -->|継承元| Domain

Infrastructure -->|継承元| Domain

開発支援ツール

- frontend ディレクトリに移動

```
pnpm typedoc
```

- TSDoc 形式で書かれた ts ファイルのコメントが反映される
- [TSDoc 形式とは](#)
- 自動でドキュメントを生成する

デザインガイドライン/デザインシステム

- UX を担保するために良いデザインの客観的基準を決める必要がある。
- デザインにセマンティックな効果（機能）をもたせる。→ デザインの意味をいちいち考える

。 カラー

- [デジタル庁デザインガイドライン カラーパレット](#)
- [Bootstrap5 ユーティリティカラー](#)
- グレースケールに関しては明度を 10 段階に分けたものを使用する。
- グレーの代表的な色を `#808080` とする。

```
color-gray-10: #1a1a1a;  
color-gray-20: #333333;  
color-gray-30: #4d4d4d;  
color-gray-40: #666666;  
color-gray-50: #808080;  
color-gray-60: #999999;  
color-gray-70: #b3b3b3;  
color-gray-80: #cccccc;  
color-gray-90: #e6e6e6;
```

- 基本的に文字は白・グレー・黒のどれかにする。→ 文字に彩度はなるべく入れない
- 本文テキストは 背景とのコントラスト比 4.5:1 以上

セマンティックカラー

- 色に役割（機能）をもたせる仕組み(装飾目的で使わない)
 - primary テーマカラー → そのアプリのブランドカラーとしての役割を持つ
 - secondary サブカラー → テーマカラーを補助する役割を持つ、primary と似た色は控える
 - success → 正常状態を表す色 基本グリーン系、処理が正常/良好に行われていることを示し、安心感を与える。「電車は予定通り運行中」
 - warning → 中立的なイメージかつ注目状態を示す。オレンジ系? 「黄色い線の内側にお並びください」
 - danger → 危険な状態/取り返しがつかない状態/絶対に気づかないといけない状態「データを削除しますか。この動作は取り消しできません」

。 フォント

- NotoSansJP を使用する
- 4px の倍数で作成する → 実際には無理

- 余白
 - 4px の倍数で作成する
 - まずはボックスレイアウト配置(左右上下余白を揃えるかつ等間隔)を心がける
- 判断に迷った場合は以下を優先する：
 1. 誤操作を防げるか
 2. 初見ユーザーが理解できるか
 3. 一貫性が保たれているか
 4. 実装が単純か